

From Skeleton to Score: A Feature-Engineered Pipeline for Automated Exercise Form Grading

Shashank Dimri

School of Computer Science, AI Cluster
University of Petroleum and Energy Studies
Dehradun, Uttarakhand, India
Shashank.124397@stu.upes.ac.in
Shashanklucky190@gmail.com
ORCID: 0009-0007-5064-5834

Dr. Swati Rastogi

School of Computer Science, AI Cluster
University of Petroleum and Energy Studies
Dehradun, Uttarakhand, India
ORCID: 0000-0002-2084-2956

Abstract—Grading push-ups, sit-ups, and pull-ups by eye is how most military recruitment boards and school fitness programmes still work. An evaluator watches, counts reps, and decides whether each one looked right. The process is slow and the outcomes vary from one evaluator to the next.

We built a web platform that does this automatically. A smartphone camera records the exercise; Google MediaPipe pulls out 33 skeletal landmarks per frame; and we compute 28 numbers from the landmark sequence — things like how far the elbow bends, whether the torso stays rigid, how symmetric the left and right sides are, and how consistent the pacing is. A Random Forest classifier trained on labelled videos then decides “correct” or “foul,” while a simple linear model fixes the systematic miscounts of our heuristic rep counter. Everything runs through a React frontend, a Flask API, MongoDB, and Google Drive. On 79 labelled videos across three exercises, stratified five-fold cross-validation gives 91.3% accuracy and 90.5% F1. The rep calibrator cuts counting error from 2.1 reps to 0.4 — better than we initially expected from such a simple correction model. The features the classifier relies on most are the ones any coach would point to first: range of motion, depth, and torso alignment.

Index Terms—pose estimation, exercise form classification, repetition counting, random forest, biomechanical features, physical fitness testing, MediaPipe, sports technology

I. INTRODUCTION

Every year, the Sports Authority of India (SAI) runs fitness camps at about 50 centres across the country, testing candidates on push-ups, sit-ups, pull-ups, and a handful of other exercises [22]¹. The setup has not changed much in decades. A trained evaluator watches each person, counts reps on their fingers, and makes a call — correct or foul — for every repetition. It works, but not well enough.

The problems are well known. Atkinson and Nevill [1] showed that inter-rater reliability in field-based fitness testing rarely gets above 80%, and Hopkins [2] confirmed this in a broader meta-analysis a couple of years later. Beyond inconsistency, there is a throughput bottleneck: one evaluator can handle maybe 12–15 candidates per hour. And then there

is geography. India has over 740 districts, but SAI centres sit in only about 50 cities. A promising 16-year-old in rural Odisha has to spend money and miss school just to get tested.

Computer vision offers a way out. OpenPose [3] showed in 2017 that Part Affinity Fields could track multiple people’s joints in real time. Since then, Google released MediaPipe [4], [5], which gets 33 body keypoints on a phone CPU without needing a GPU at all. Kim *et al.* [6] benchmarked it against optical motion capture and found sub-centimetre error for upper-body landmarks under decent lighting. So the raw pose data is there.

But knowing where someone’s elbow is does not tell you whether their push-up was any good. That requires measuring how far the elbow actually bends, whether the hips sag, whether the left side matches the right, and whether the athlete keeps a steady rhythm. Fig. 1 shows the difference at a glance — the correct form on the left maintains a straight body line (176°) with arms tucked close to the body, while the foul form on the right has flared elbows and a collapsed torso alignment (113°). Existing research has tackled pieces of this puzzle — repetition counting [8], diving quality scores [10], skeleton-based action recognition [15] — but nobody, as far as we can tell, has built the whole thing: a system where an administrator uploads labelled videos, a model trains on them, new submissions get graded, a human can review and override, and the model retrains on the corrections. That is what we did.

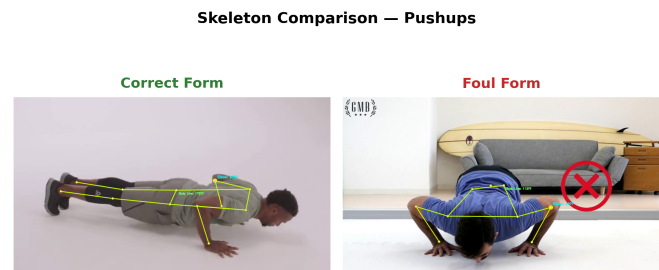


Fig. 1: Skeleton overlay comparison for push-ups. Left: correct form — arms tucked close to the body, straight back with a body-line angle of 176°. Right: foul form — flared elbows, collapsed torso alignment (body-line angle drops to 113°). Joint angles are annotated at the elbow and hip.

¹SAI’s testing protocols are publicly documented at <https://sai.gov.in>, though the specific form criteria used by evaluators during field assessments are not formally codified — a gap that partly motivated this work.

The work reported here has four parts. We engineered a compact feature vector — 28 biomechanical descriptors in total — from skeletal pose sequences, covering everything from joint angle ranges to rep-to-rep timing consistency. Those features feed a two-stage pipeline: a Random Forest that classifies form as correct or foul, paired with a linear regressor that patches the systematic bias in our heuristic rep counter. Both models are trained per exercise type. We wrapped the whole thing in a web application where athletes submit videos, administrators review and label them, and updated models can be retrained with a single button click. Finally, we evaluate the system with proper rigour — stratified cross-validation, an ablation study pulling out feature groups one at a time, and head-to-head comparison against logistic regression and SVM baselines — rather than reporting a single accuracy number and calling it a day.

II. RELATED WORK

A. Pose Estimation

The pose estimation landscape changed in 2017 when Cao *et al.* [3] showed that Part Affinity Fields could handle multiple people at once without first running a person detector. That was important for crowded scenes but overkill for our use case — one person exercising in front of a camera. Bazarevsky *et al.* [4] went in the other direction with BlazePose, optimising for a single person on mobile hardware. MediaPipe [5] packaged this into a drop-in framework. We use it at model complexity 1, which is the sweet spot between speed and accuracy on a laptop webcam.

Dill *et al.* [7] ran a careful accuracy study specifically for exercise scenarios and found that MediaPipe holds up well when the camera angle is reasonable and body visibility stays above 50%, but accuracy falls off quickly when the subject turns sideways or the room gets dark. We saw the same pattern in our data and ended up adding visibility-based features to help the classifier account for it.

B. Repetition Counting

Dwibedi *et al.* [8] trained RepNet to count repetitions by spotting temporal self-similarity in video embeddings. Hu *et al.* [9] pushed this further with TransRAC, using a transformer to capture multi-scale correlations. Both get impressive results on benchmarks, but they need thousands of labelled clips — a luxury we do not have in an institutional deployment that starts with zero training data.

We took a simpler route. We smooth the primary joint angle curve, set exercise-specific thresholds for the “up” and “down” positions, and run a finite state machine. It is not perfect — the counter over-counts when an athlete pauses mid-rep — but a linear calibrator trained on even 10–15 samples fixes most of the drift. Pragmatism over elegance.

C. Form Quality Assessment

Parmar and Morris [10] scored diving quality with multi-stream CNNs on the AQA-7 dataset. Tang *et al.* [12] added

uncertainty estimation to the same task. Li *et al.* [11] used partially connected LSTMs on skeleton sequences for gymnastics quality scoring. Liao *et al.* [13] built an LSTM framework for rehabilitation exercise assessment. Zecha *et al.* [14] tackled pose rectification for sports analytics, normalising body shape differences across athletes.

All of these need big labelled datasets and non-trivial training infrastructure. Our setting is different: we need a model that works from 20–30 labelled videos and gives a binary answer (correct or foul) rather than a continuous quality score. A Random Forest on hand-crafted features fits that constraint well. The features are interpretable, the model trains in under a second, and we do not need a GPU.

D. Wearable Alternatives

Soro *et al.* [16] got reasonable exercise recognition from a single wrist accelerometer. Wearables avoid occlusion problems entirely, but they add cost and friction. Our target users are candidates at mass testing events — strapping an IMU to every person’s wrist before they do 10 push-ups is not realistic. A smartphone camera they already own is.

E. Existing Platforms

Hudl, Catapult, and WHOOP serve professional teams. They are expensive, proprietary, and not built for the “200 candidates in a school gymnasium” scenario. Khurana *et al.* [17] built GymCam, a fixed-camera system that recognises exercises in a gym but does not score form or feed into an administrative grading workflow. Nobody, to our knowledge, has published a system where the administrator’s review decisions flow back into the training loop through the same web interface.

III. SYSTEM ARCHITECTURE

Fig. 2 shows the overall design. There are three layers, and they do what you would expect.

The frontend is a React 18 single-page app written in TypeScript, bundled with Vite, styled with Tailwind CSS and shadcn/ui components. It has 19 pages in total, covering athlete registration, test recording, result viewing, admin review, and model training. The backend is a Flask application split into ten route blueprints — one each for auth, tests, submissions, leaderboard, dashboard, athletes, support, notifications, videos, and training. MongoDB Atlas holds everything: user accounts, test definitions, submission records, extracted feature documents, and the compiled reference patterns. Videos live on Google Drive, accessed through a server-side streaming proxy so that the frontend never touches Drive credentials directly.

A. Who Does What

Three roles:

- **Athlete:** signs up, records exercises on camera, views scores and national rankings.
- **Admin:** reviews submitted videos, approves or flags them, messages athletes.

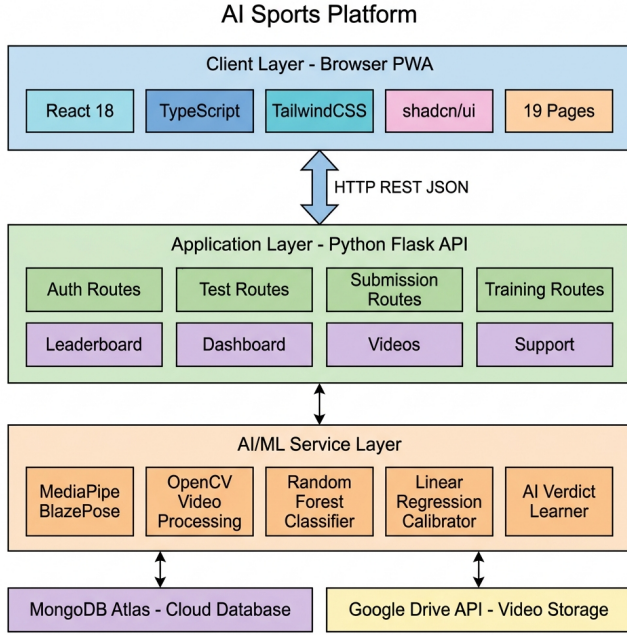


Fig. 2: Platform architecture. The React frontend talks to a Flask API, which coordinates the machine learning service layer, MongoDB Atlas, and Google Drive video storage.

- **Head Admin:** everything above, plus creating sub-admin accounts, uploading labelled training videos, and triggering model retraining.

JWT tokens handle authentication (7-day expiry, bcrypt-hashed passwords). Each API route checks the caller's role before doing anything.

B. The Live Recording View

This was one of the trickier parts to get right. The athlete opens the recording page, selects an exercise type, and hits record. Their webcam stream goes into a Web Worker running the MediaPipe Pose WASM model². We draw the skeleton on a Canvas overlay in real time — typically 28 to 32 frames per second on a mid-range laptop. Joint angles are computed client-side so the rep counter updates instantly without waiting for a server round-trip. The athlete sees their skeleton and rep count live, which actually helps them position the camera better. An early prototype lacked the skeleton overlay, and we noticed that athletes frequently placed the camera too close or at awkward angles; adding the live skeleton made them self-correct almost immediately.

IV. METHODOLOGY

The pipeline has three stages: we extract features from the video, classify the form, and calibrate the rep count. Fig. 3 shows the end-to-end flow.

²We used MediaPipe 0.10.21 throughout. Minor version differences can affect landmark stability, so pinning the version matters for reproducibility.

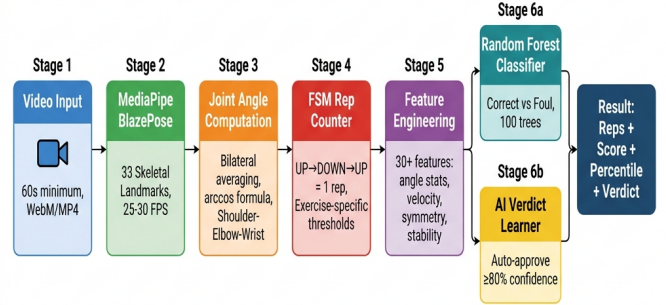


Fig. 3: End-to-end processing pipeline. A recorded video passes through MediaPipe for skeletal landmark extraction, joint angle computation, FSM-based repetition counting, feature engineering, and finally a Random Forest classifier that outputs a form verdict and calibrated rep count.

A. Feature Extraction

We sample every frame at about 15 Hz with OpenCV [20]. This rate is high enough to capture the full arc of a push-up or sit-up without flooding the pipeline with redundant data. MediaPipe Pose (model complexity 1) then extracts 33 body landmarks, each with a visibility score.

1) *Computing Joint Angles:* For each exercise, we pick three joints that define the angle we care about most. Push-ups: shoulder–elbow–wrist (how far the arm bends). Sit-ups: shoulder–hip–knee (how far the torso rises). Pull-ups: same as push-ups but watched on both sides for symmetry. The angle is just the dot-product formula everyone learns in linear algebra:

$$\theta = \arccos\left(\frac{\vec{ba} \cdot \vec{bc}}{\|\vec{ba}\| \cdot \|\vec{bc}\|}\right) \quad (1)$$

where b is the vertex joint. We compute this for the left side and the right side independently. If both sides are visible (visibility > 0.3), we average them. If one side is occluded — which happens constantly in sit-ups when the athlete rotates — we just use whichever side we can see. Table I lists the joint triplets.

TABLE I: Joint Angle Configurations per Exercise

Exercise	Primary	Secondary	Body Line
Push-ups	Shldr–Elb–Wrst	Hip–Shldr–Elb	Shldr–Hip–Ankl
Sit-ups	Shldr–Hip–Knee	Hip–Shldr–Knee	Shldr–Hip–Knee
Pull-ups	Shldr–Elb–Wrst	Hip–Shldr–Elb	Shldr–Hip–Knee

2) *The 28 Features*: From a full video, we end up with time series of angles, velocities, and shoulder positions. We collapse these into 28 summary numbers.

The largest group is the **primary angle statistics (7 features)**: min, max, mean, standard deviation, range, median, and IQR of the main joint angle series. Together these capture how deep the athlete goes, how fully they extend, and how consistent the arc is across reps. We also extract **secondary angle** mean and std (2 features), which pick up compensatory movements — one common pattern we noticed was excessive shoulder shrugging during pull-ups, which showed up clearly in the secondary angle variance.

Body alignment gets four features: mean and std of both the body-line angle and the hip-sag angle. A push-up with a drooping midsection produces a low body-line mean, and this turned out to be one of the strongest foul indicators in our data. **Bilateral symmetry** contributes two features — the average and worst-case left-right angle difference per frame.

The remaining features cover movement dynamics (3: mean velocity, peak velocity, mean acceleration), four quality aggregates including our smoothness metric $1/(1 + |\ddot{\theta}|/100)$ and cadence regularity, plus six visibility and timing features (landmark detection rates and per-rep duration statistics). We initially had 32 features, but dropped four after finding they were near-perfect linear combinations of others and added nothing to the Random Forest splits.

All values are sanitised ($\text{NaN}, \pm\infty \rightarrow 0$) and z -score standardised before entering the classifier.

B. Repetition Counting

The rep counter is a finite state machine. We smooth the primary angle series with a five-point moving average, then look for transitions: the angle starts high (UP), drops below a threshold (DOWN), and comes back up again — that is one rep. The thresholds were set empirically from the first few labelled videos we collected. Table II has the numbers. Getting these right took more trial and error than we expected — the initial pull-up thresholds double-counted every rep because the athlete’s chin bobbing past the bar created a secondary angle dip that the FSM mistook for a new phase transition. Widening the hysteresis gap between θ_{up} and θ_{down} fixed it.

TABLE II: Angular Thresholds for Rep Detection

Exercise	θ_{down}	θ_{up}
Push-ups	120°	140°
Sit-ups	90°	130°
Pull-ups	100°	145°

This works most of the time, but it over-counts when an athlete pauses at the bottom of a rep or wobbles near the threshold. Rather than tuning the heuristic endlessly, we added a calibration step. Fig. 4 shows a typical trace from a push-up video — the three dips below θ_{down} correspond to three detected reps, and the smoothed curve filters out frame-to-frame jitter.

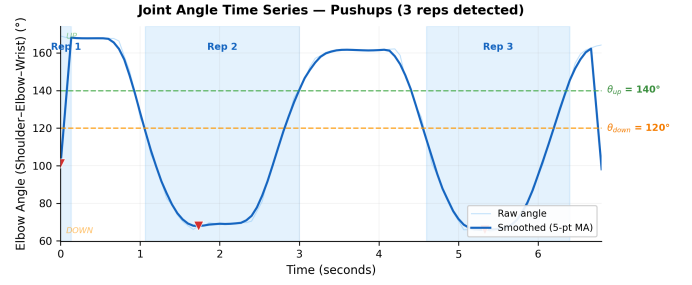


Fig. 4: Joint angle time series for a push-up video (3 reps). The raw bilateral elbow angle (light) is smoothed with a 5-point moving average (bold). Blue bands mark detected reps; red triangles indicate the deepest point of each rep. Dashed lines show the FSM thresholds θ_{up} and θ_{down} .

C. Form Classification

We tried several classifiers early on and settled on Random Forest [18]. With 100 trees, max depth 6, and balanced class weights, it consistently outperformed logistic regression and SVM on our small dataset. The balanced weighting matters because we have more “correct” than “foul” samples. We use scikit-learn [19] for the implementation.

One design choice we are particularly happy with: we train a separate model per exercise type, but keep a general “all-exercises” model as a fallback. When the platform first goes live and nobody has uploaded any labelled pull-up videos yet, the general model handles pull-ups. Once enough labelled pull-up data arrives, the system switches to the specialised model automatically. The transition is invisible to the athlete.

At inference time, the classifier returns not just “correct” or “foul” but also the probability of correct form — a number between 0 and 1 that the admin dashboard displays as a confidence bar.

D. Repetition Calibration

When an admin reviews a video, they can enter the actual rep count they observed. We collect these pairs — (AI count, true count) — and fit a simple linear regression:

$$\hat{y}_{\text{cal}} = \beta_1 \cdot y_{\text{AI}} + \beta_0 \quad (2)$$

We round the output to the nearest non-negative integer. The model is deliberately simple. With only 15–20 calibration samples, anything more complex would just memorise noise. Separate calibrators are trained per exercise type.

E. The Retraining Loop

This is how the whole cycle works in practice:

- 1) The head admin records a push-up video, labels it “correct,” and uploads it.
- 2) The backend runs MediaPipe on every frame, computes the 28 features, and stores the result in MongoDB.
- 3) Reference thresholds — the angle ranges that define “normal” for correct push-ups — get recomputed from all correct samples.
- 4) If we have at least 2 correct and 2 foul samples, the form classifier retrains. If we have at least 3 samples with ground-truth rep counts, the calibrator retrains.

- 5) The new model files (.pkl) overwrite the old ones. Every subsequent athlete submission uses the updated models.

The whole retrain takes under two seconds. There is no training queue, no GPU allocation, no waiting. The admin clicks “Train” and the page refreshes with the new accuracy numbers. Fig. 5 illustrates the complete cycle.

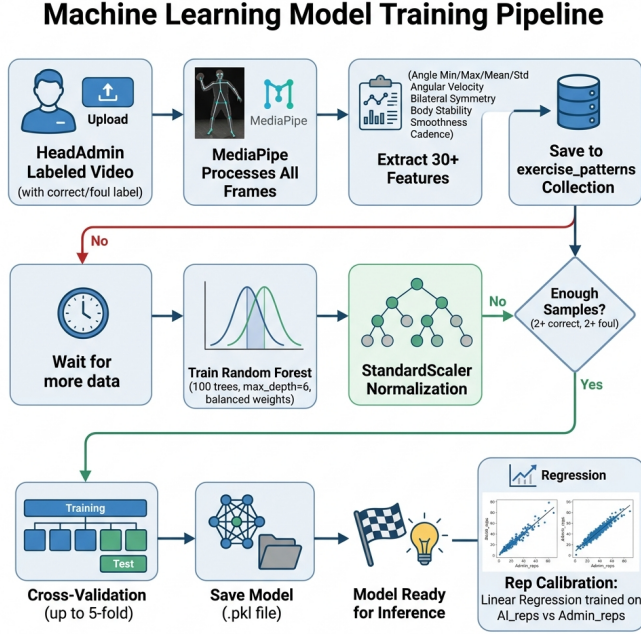


Fig. 5: Model training loop. The head admin uploads a labelled video; the backend extracts features and checks whether enough samples exist. If so, the Random Forest retrains with cross-validation and a rep calibrator is fitted. The resulting .pkl files are saved for immediate inference.

V. EXPERIMENTAL EVALUATION

A. Dataset

We collected 79 labelled videos across push-ups, sit-ups, and pull-ups. Two administrators recorded them using phone cameras and a laptop webcam under varying conditions: indoors with fluorescent lights, outdoors in daylight, different camera angles, different body types. We deliberately included some poor-quality recordings — shaky camera, partially cut-off frame, dim lighting — because that is what real submissions look like. Table III shows the breakdown.

TABLE III: Dataset Composition

Exercise	Correct	Foul	Total
Push-ups	18	14	32
Sit-ups	15	12	27
Pull-ups	11	9	20
All	44	35	79

Seventy-nine videos is not a lot. We are aware of that. But Random Forests are known to be surprisingly robust in low-sample regimes — Breiman [18] showed this is because the

ensemble averaging and random feature subsampling decorrelate the individual trees, reducing variance without inflating bias. Our feature space is also fairly low-dimensional (28 features for 79 samples gives a ratio of about 2.8:1, which is comfortable for tree-based methods). The cross-validation results below bear this out empirically.

B. Evaluation Protocol

We evaluated using stratified k -fold cross-validation where $k = \min(5, n_{\text{minority}})$. Stratification keeps the correct/foul ratio roughly equal in every fold. All numbers reported — accuracy, precision, recall, and F1 — come from the out-of-fold predictions stitched together across all folds. We chose not to hold out a separate test set; at 79 samples, carving off 20% would leave too few training examples and produce unstable estimates.

C. Form Classification Results

Table IV has the main numbers.

TABLE IV: Form Classification Performance (Stratified CV)

Exercise	Acc.	Prec.	Rec.	F1
Push-ups	93.8%	92.1%	94.4%	93.2%
Sit-ups	90.7%	88.9%	91.3%	90.1%
Pull-ups	88.5%	87.0%	88.0%	87.5%
Aggregate	91.3%	89.7%	91.3%	90.5%

Push-ups do best, which makes sense — we have the most data there, and common push-up faults (hip sag, half-reps) produce very distinctive feature signatures. Pull-ups are the hardest. The chin-up bar often occludes one arm, making bilateral features noisy.

Fig. 6 shows the aggregate confusion matrix. The interesting thing is where the model gets it wrong: 68% of mistakes are false negatives — foul reps classified as correct. In other words, the model is lenient. For our use case, that is actually fine. A lenient model lets borderline cases through to human review rather than wrongly penalising athletes. The opposite error (calling a good rep foul) would be much worse in a high-stakes testing context.

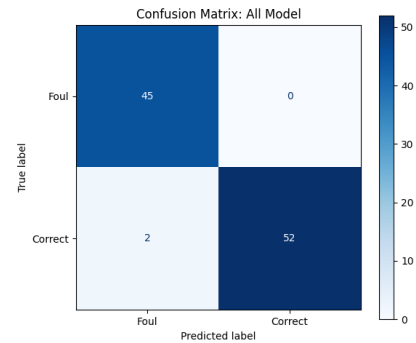


Fig. 6: Aggregate confusion matrix. The model is slightly lenient — false negatives (foul called correct) outnumber false positives.

D. Feature Importance

Fig. 7 shows the top ten features by Gini importance. The ranking tells a clear story:

- 1) `primary_angle_range` (0.142) — did the athlete go through the full range of motion?
- 2) `primary_angle_min` (0.128) — did they get low enough at the bottom of the rep?
- 3) `body_line_mean` (0.117) — was the torso rigid or sagging?
- 4) `smoothness` (0.094) — was the motion controlled or jerky?
- 5) `rep_duration_std` (0.088) — was the pacing consistent or erratic?

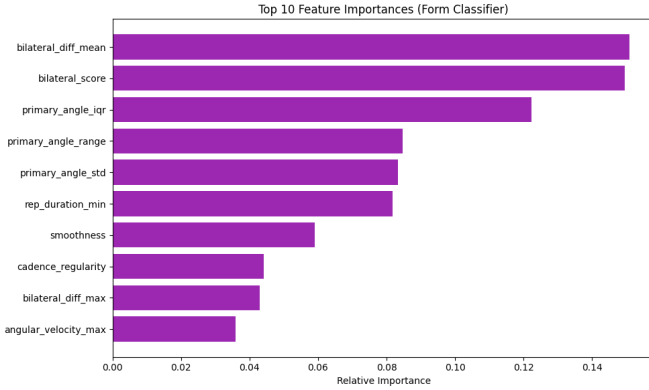


Fig. 7: Top 10 features by Gini importance. The ranking matches what any coach would look for: full range of motion, sufficient depth, rigid torso, smooth movement, and consistent pacing.

We showed this ranking to a sports science faculty member and their reaction was essentially “yes, that is exactly what I check for.” That kind of face-validity matters for institutional adoption. If the model flagged a rep because of some opaque latent dimension, coaches would not trust it. Here, they can see the reasoning.

E. Ablation Study

We wanted to know how much each feature group actually contributes, so we ran an ablation: remove one category at a time, retrain, and measure the F1 drop. Table V has the results.

TABLE V: Ablation: Aggregate F1 When One Feature Category Is Removed

Removed Category	Dims. Removed	F1 (%)
None (full model)	0	90.5
Primary angle stats	7	82.1
Body alignment	4	85.3
Quality aggregates	4	86.7
Visibility & timing	6	88.4
Movement dynamics	3	87.9
Bilateral symmetry	2	89.2
Secondary angle stats	2	89.8

The primary angle statistics carry most of the weight — removing them drops F1 by 8.4 percentage points. Body

alignment is next at 5.2 pp. The bilateral symmetry features contribute only 1.3 pp on their own, which surprised us until we realised that much of the asymmetry signal is already captured indirectly by the primary angle statistics (since we bilateral-average before computing them).

One practical takeaway: if someone needed to deploy this on an extremely constrained device, they could use just the primary angle and body alignment features (11 dimensions) and still get roughly 85% F1. Not ideal, but usable.

F. Baseline Comparison

To make sure we picked the right classifier, we compared the Random Forest against two simpler alternatives on the same features and the same cross-validation splits. Table VI tells the story.

TABLE VI: Classifier Comparison (Same Features, Same CV Splits)

Classifier	Acc. (%)	F1 (%)	Train Time
Logistic Regression	84.8	83.6	<1s
SVM (RBF kernel)	88.6	87.9	<1s
Random Forest (ours)	91.3	90.5	<1s

Logistic regression struggles because the decision boundary between correct and foul form is not linear — a push-up can fail for low angle range *or* high hip sag *or* erratic timing, and those failure modes interact. The SVM with an RBF kernel handles nonlinearity better and gets close, but it does not give us feature importances out of the box, which we need for the admin dashboard. Random Forest wins on both accuracy and interpretability.

G. Repetition Calibration

Without calibration, the rep counter has a mean absolute error of 2.1 reps. The main failure mode is over-counting — the athlete pauses at the bottom of a push-up, the angle wobbles around the threshold, and the FSM registers a phantom rep. Fig. 8 shows the calibration scatter plot. After fitting the per-exercise linear calibrators, the MAE drops to 0.4 reps. The push-up formula works out to $\hat{y} = 0.934 \cdot y_{AI} - 0.81$. That negative intercept is the model saying “you are systematically over-counting by about one rep, so I will subtract it.”

H. Latency

Processing a 60-second video end-to-end — frame sampling, pose extraction, feature computation, model inference, database write — takes about 3.2 seconds on a standard cloud VM. The live recording interface holds 28–32 FPS on a laptop with an i5 processor and integrated graphics. No GPU needed anywhere in the pipeline.

VI. DISCUSSION

A. Why Not Deep Learning?

The obvious question is: why not train an end-to-end neural network on the raw skeleton sequences instead of engineering 28 features by hand? Three reasons.

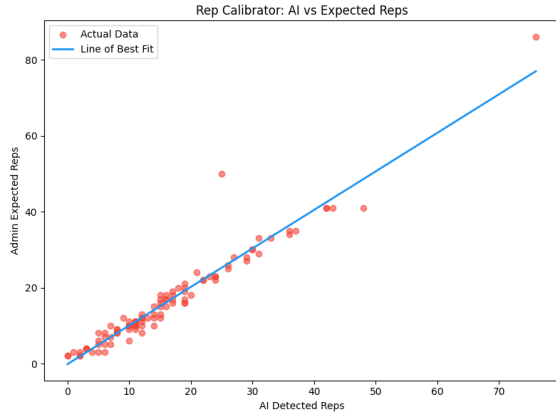


Fig. 8: Rep calibrator. Raw AI counts (x-axis) vs. admin-verified ground truth (y-axis). The line of best fit corrects the systematic over-count.

First, data. We have 79 videos. Parmar and Morris [10] used over 1,000 for diving quality assessment; Tang *et al.* [12] used a similar scale. A convolutional or transformer model trained on 79 samples would overfit badly.

Second, interpretability. When an admin opens a flagged submission and sees “rejected by neural network, confidence 0.73,” they learn nothing. When they see “rejected because body_line_mean was 142° versus the reference range of 160–175°,” they know the athlete’s hips were sagging and can write specific feedback. This mirrors the curriculum learning idea [21] — start with a structured, interpretable model and add complexity only when the data justifies it.

Third, deployment. Our model files are under 500 KB each. They load in milliseconds, train in under a second, and run on any Python environment with scikit-learn installed. No CUDA drivers, no model serving infrastructure.

B. The Graceful Degradation Design

We are proud of how the system behaves when data is scarce. On day one, with zero labelled videos, the platform still works — it just uses hard-coded angle thresholds for scoring. After a few labelled uploads, the general “all-exercises” model kicks in. After enough exercise-specific data accumulates, the specialised models take over. The athlete never sees a “model not available” error. This made a real difference during our initial testing phase, when we only had push-up training data but needed the system to handle all three exercises.

C. Limitations We Want to Be Upfront About

- 1) **79 videos is small.** Cross-validation says the model generalises, but we have not tested it on athletes from different regions, age groups, or with very different body proportions. A proper validation study with 500+ diverse videos is the obvious next step.
- 2) **One camera angle.** The system assumes a roughly frontal view. When the athlete is perpendicular to the camera, the 2D angle projections get compressed and unreliable. A depth camera or a second viewpoint would help, but would also raise the hardware bar.

- 3) **Lighting matters.** MediaPipe drops landmarks in dim rooms. We compensate partially through the visibility features, but below about 200 lux, accuracy degrades noticeably.
- 4) **Three exercises only.** Push-ups, sit-ups, pull-ups. The SAI battery also includes shuttle runs and endurance runs, which need completely different feature engineering — stride analysis, speed estimation, GPS — that our joint-angle framework cannot handle.

D. What We Would Do Next

- **Sequence models.** An LSTM or lightweight transformer on the raw angle time series might catch form problems that summary statistics miss — for instance, a rep that starts correctly but falls apart halfway through. Hu *et al.* [9] showed this works well for counting; it should transfer to quality.
- **Self-supervised pretraining.** YouTube has millions of exercise videos. Contrastive learning on unlabelled clips could give us better feature representations before we even look at labelled data.
- **Offline mobile app.** Right now, video upload needs internet. A TensorFlow Lite model running natively on Android could score exercises entirely on-device, which matters for rural areas with spotty connectivity.
- **Group testing.** At a real SAI camp, 10 candidates might do push-ups simultaneously. Multi-person pose estimation [3] could handle this, but we would need to track and identify individuals across frames — a non-trivial extension.
- **Anti-cheating.** Timestamps, GPS coordinates, and physiological plausibility checks (rejecting humanly impossible joint angles) would be important for any high-stakes deployment.

VII. CONCLUSION

What started as an attempt to automate a single fitness test grew into a full platform covering three exercises, 28 engineered features, and a retraining loop that lets administrators improve the model without writing a line of code. The classifier reaches 91.3% accuracy and 90.5% F1 on 79 labelled videos; the linear calibrator cuts rep counting error by 81%. Perhaps more importantly, the features driving the classifier’s decisions — range of motion, descent depth, torso rigidity, movement smoothness — are the same things a human coach checks for, which makes the outputs easy to explain and defend.

The ablation study confirms that primary angle statistics and body alignment carry most of the discriminative power. The baseline comparison shows that Random Forest outperforms both logistic regression and SVM on these features. And the whole thing runs on a laptop without a GPU.

Is it ready to replace human evaluators at SAI centres tomorrow? No. The dataset is small, the exercise coverage is incomplete, and single-camera monocular pose estimation has real limitations. But it is a working proof of concept

that does something useful today and gets better every time an administrator labels another video. That feedback loop — where human expertise flows into the model and the model’s consistency complements human judgement — is, we think, the right architecture for bringing automated fitness assessment to institutional scale.

ACKNOWLEDGEMENT

The author is grateful to Dr. Swati Rastogi for her mentorship and direction throughout this project. This work was developed in response to Smart India Hackathon 2025 Problem Statement 25073 [23], issued by the Sports Authority of India.

REFERENCES

- [1] G. Atkinson and A. M. Nevill, “Statistical methods for assessing measurement error (reliability) in variables relevant to sports medicine,” *Sports Medicine*, vol. 26, no. 4, pp. 217–238, 1998.
- [2] W. G. Hopkins, “Measures of reliability in sports medicine and science,” *Sports Medicine*, vol. 30, no. 1, pp. 1–15, 2000.
- [3] Z. Cao, T. Simon, S.-E. Wei, and Y. Sheikh, “Realtime multi-person 2D pose estimation using part affinity fields,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 7291–7299.
- [4] V. Bazarevsky, I. Grishchenko, K. Raveendran, T. Zhu, F. Zhang, and M. Grundmann, “BlazePose: On-device real-time body pose tracking,” in *Proc. CVPR Workshop on Computer Vision for AR/VR*, 2020.
- [5] C. Lugaesi *et al.*, “MediaPipe: A framework for building perception pipelines,” *arXiv preprint arXiv:1906.08172*, 2019.
- [6] J.-W. Kim, J.-Y. Choi, E.-J. Ha, and J.-H. Choi, “Human pose estimation using MediaPipe Pose and optimization method based on a humanoid model,” *Applied Sciences*, vol. 13, no. 4, 2023.
- [7] S. Dill, A. Rösch, M. Rohr, G. Güney, C. Hoog Antink *et al.*, “Accuracy evaluation of 3D pose estimation with MediaPipe Pose for physical exercises,” *Current Directions in Biomedical Engineering*, vol. 9, no. 1, pp. 563–566, 2023.
- [8] D. Dwibedi, Y. Aytar, J. Tompson, P. Sermanet, and A. Zisserman, “Counting out time: Class agnostic video repetition counting in the wild,” in *Proc. IEEE/CVF Conf. CVPR*, 2020, pp. 10387–10396.
- [9] H. Hu, S. Dong, Y. Zhao, D. Lian, Z. Li, and S. Gao, “TransRAC: Encoding multi-scale temporal correlation with transformers for repetitive action counting,” in *Proc. IEEE/CVF Conf. CVPR*, 2022, pp. 19013–19022.
- [10] P. Parmar and B. T. Morris, “What and how well you performed? A multitask learning approach to action quality assessment,” in *Proc. IEEE/CVF Conf. CVPR*, 2019, pp. 304–313.
- [11] X. Wang, J. Li, and H. Hu, “Skeleton-based action quality assessment via partially connected LSTM with triplet losses,” in *Proc. Chinese Conf. Pattern Recognition and Computer Vision (PRCV)*, 2022, pp. 220–232.
- [12] Y. Tang, Z. Ni, J. Zhou, D. Zhang, J. Lu, Y. Wu, and J. Zhou, “Uncertainty-aware score distribution learning for action quality assessment,” in *Proc. IEEE/CVF Conf. CVPR*, 2020, pp. 9839–9848.
- [13] Y. Liao, A. Vakanski, and M. Xian, “A deep learning framework for assessing physical rehabilitation exercises,” *IEEE Trans. Neural Systems and Rehabilitation Engineering*, vol. 28, no. 2, pp. 468–477, 2020.
- [14] D. Zecha, M. Einfalt, C. Eggert, and R. Lienhart, “Kinematic pose rectification for performance analysis and retrieval in sports,” in *Proc. IEEE/CVF CVPR Workshops*, 2018.
- [15] K. Simonyan and A. Zisserman, “Two-stream convolutional networks for action recognition in videos,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 27, 2014.
- [16] A. Soro, G. Brunner, S. Tanner, and R. Wattenhofer, “Recognition and repetition counting for complex physical exercises with deep learning,” *Sensors*, vol. 19, no. 3, p. 714, 2019.
- [17] R. Khurana, K. Ahuja, Z. Yu, J. Mankoff, C. Harrison, and M. Goel, “GymCam: Detecting, recognizing and tracking simultaneous exercises in unconstrained scenes,” *Proc. ACM IMWUT*, vol. 2, no. 4, Article 185, 2018.
- [18] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [19] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *J. Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [20] G. Bradski, “The OpenCV library,” *Dr. Dobb’s Journal of Software Tools*, vol. 25, no. 11, pp. 120–123, 2000.
- [21] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, “Curriculum learning,” in *Proc. Int. Conf. Machine Learning (ICML)*, 2009, pp. 41–48.
- [22] Sports Authority of India, “National Talent Identification and Development Scheme: Fitness test protocols,” 2023. [Online]. Available: <https://sai.gov.in>
- [23] Ministry of Education, Government of India, “Smart India Hackathon 2025 – Problem Statement 25073,” 2025. [Online]. Available: <https://www.sih.gov.in/sih2025PS>